

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Centro Universitario de Chimaltenango

Curso: Sistemas Operativos I · Sección: A

Catedrático: _____

PROYECTO 2

Motor de Transacciones Concurrentes y Detector de Interbloqueos

BiblioSync — Sistema de préstamo de libros de una biblioteca universitaria con varias sucursales

Unidades 3 (Concurrencia y Sincronización) y 4 (Interbloqueos)

Documentación: propuesta (Entregable 0), marco teórico, diseño, evidencias y manual de ejecución

Integrantes del equipo

No.	Nombre completo del integrante	Carné
1		
2		
3		
4		

Chimaltenango, septiembre de 2026

Índice

1. Introducción.....	4
2. Objetivos.....	4
2.1 Objetivo general.....	4
2.2 Objetivos específicos.....	4
3. Marco teórico.....	6
3.1 Procesos e hilos.....	6
3.2 Hilos en Linux: modelo 1:1, NPTL y /proc.....	6
3.3 Hilos en Python: threading y el GIL.....	6
3.4 Concurrencia, sección crítica y condición de carrera.....	7
3.5 Mecanismos de sincronización.....	7
3.6 Transacciones, bloqueo en dos fases y bitácora.....	8
3.7 Interbloqueos y condiciones de Coffman.....	9
3.8 Grafo de Asignación de Recursos (RAG).....	9
3.9 Estrategias para manejar interbloqueos.....	10
3.10 Algoritmos de detección: DFS, Tarjan y Work/Finish.....	10
3.11 Recuperación: víctima, rollback e inanición.....	11
3.12 Comunicación en tiempo real con el navegador.....	11
4. Entregable 0 — Propuesta del proyecto.....	12
4.1 Escenario elegido y por qué genera contención real.....	12
4.2 Lenguaje, entorno y mecanismo de sincronización.....	12
4.3 Recursos compartidos e hilos que compiten.....	13
4.4 Cómo provocaremos y comprobaremos la condición de carrera.....	13
4.5 Diseño inicial de la espera circular (modo caótico).....	14
4.6 Estrategia de detección y recuperación propuesta.....	14
4.7 Propuesta de frontend.....	14
4.8 Cronograma de trabajo hasta la entrega final.....	15
5. Diseño e implementación.....	16
5.1 Arquitectura.....	16
5.2 Módulos del repositorio.....	16
5.3 Ciclo de vida de una transacción de préstamo.....	17
5.4 El RAG se construye desde el estado real.....	17
5.5 Hilo guardián: detección y recuperación.....	18
5.6 Frontend en tiempo real.....	18
6. Entregable 1 — Motor concurrente e integridad bajo carga.....	20
6.1 Prueba de condición de carrera en el panel.....	20
6.2 Prueba de carga (100 hilos × 200 operaciones).....	21
7. Entregable 2 — Interbloqueo, RAG y recuperación.....	22
7.1 Fase 1: hora pico, cada hilo toma su primer libro.....	22
7.2 Fase 2: espera circular detectada por el watchdog.....	22
7.3 Fase 3: recuperación por rollback.....	23
7.4 Sin pausas: el interbloqueo aparece por sí solo.....	24
8. Evidencia en el sistema operativo.....	26
8.1 Hilos del proceso con su estado en el kernel.....	26
8.2 Detalle leído de /proc/<PID>/task.....	26
8.3 Bitácora (WAL) con el TID de cada evento.....	27
8.4 Otros comandos de verificación.....	27
9. Entregable 3 — Repositorio, reproducción y demostración.....	28
9.1 Estructura del repositorio.....	28
9.2 Reproducir en WSL2.....	28

9.3 Guion de la demostración en vivo.....	28
9.4 Publicación en la Wiki.....	29
10. Conclusiones.....	29
11. Referencias.....	30
Anexo A. Lista de revisión del docente.....	31

Si el índice aparece vacío o desactualizado, en Word haga clic derecho sobre él → «Actualizar campos» → «Actualizar toda la tabla».

1. Introducción

Cuando varios hilos de ejecución comparten datos, el resultado de un programa deja de depender solo de su código y pasa a depender también del **orden en que el planificador del sistema operativo intercala esos hilos**. Si ese orden no se controla aparecen dos familias de errores que estudian las Unidades 3 y 4 del curso: las **condiciones de carrera**, que corrompen los datos, y los **interbloqueos** (*deadlocks*), que detienen el sistema para siempre.

Este proyecto construye **BiblioSync**, un motor de transacciones para el sistema de préstamo de libros de una biblioteca universitaria con varias sucursales conectadas al mismo catálogo. Cada título tiene un número fijo de copias físicas y los usuarios piden préstamos combinados (dos o más libros en una misma transacción). Cada solicitud se ejecuta en un **hilo real del sistema operativo** (hilo NPTL de Linux, verificable con `ps -eLf` y `/proc/[pid]/task`) y compite por las copias protegidas con **semáforos contadores** (`threading.Semaphore`).

Sobre ese motor se demuestra, con evidencia externa al programa, que: (1) sin sincronización se produce un sobre-préstamo real; (2) con semáforos la integridad del inventario se mantiene bajo carga; (3) si se relaja el orden de adquisición de recursos surge una **espera circular genuina**; y (4) un **hilo guardián** detecta el ciclo recorriendo con DFS un **Grafo de Asignación de Recursos (RAG)** construido a partir del estado real del programa y lo resuelve abortando (rollback) a una transacción víctima. Todo se visualiza en tiempo real en un panel web alimentado por el backend mediante WebSocket.

El documento sigue paso a paso la guía «Proyecto 2: Definición del proyecto»: incluye el marco teórico, la propuesta (Entregable 0) basada en la hoja de trabajo presentada en clase, el diseño e implementación, las evidencias de los Entregables 1 y 2 y el material del Entregable 3 (repositorio, README, guion de la demostración y página para la Wiki).

2. Objetivos

2.1 Objetivo general

Implementar un motor transaccional concurrente con hilos reales del sistema operativo Linux que proteja sus secciones críticas con semáforos y locks nativos, y que incorpore un módulo detector de interbloqueos basado en un Grafo de Asignación de Recursos, con una estrategia de recuperación y una interfaz que refleje en tiempo real el estado del backend.

2.2 Objetivos específicos

1. Modelar el préstamo combinado de libros como transacciones que compiten por recursos contables (copias por ISBN y estaciones de préstamo) usando `threading.Semaphore`.
2. Provocar y medir una condición de carrera real (sobre-préstamo) y demostrar que el semáforo la elimina, incluso con 100 hilos y 20 000 operaciones.
3. Diseñar un modo caótico en el que la espera circular surja del orden de las solicitudes y del planificador, no de pausas artificiales.

4. Construir el RAG leyendo las estructuras internas del motor y detectar ciclos con DFS en un hilo guardián, confirmando el interbloqueo con el algoritmo de detección para recursos con múltiples instancias.
5. Recuperar el sistema abortando a la transacción que entró más recientemente al ciclo, con rollback basado en una bitácora (WAL) y reintento sin inanición.
6. Presentar evidencias externas (`ps -eLf`, `/proc/[pid]/task`, `htop`, `strace`) y un panel web que muestre hilos, recursos, grafo y registros del kernel.

3. Marco teórico

3.1 Procesos e hilos

Un **proceso** es un programa en ejecución: tiene su propio espacio de direcciones, archivos abiertos, credenciales y al menos un flujo de control. El sistema operativo lo representa con un bloque de control de proceso (PCB). Un **hilo** (*thread*) es la unidad básica de utilización de la CPU dentro de un proceso: posee su propio identificador, contador de programa, conjunto de registros y pila, pero **comparte** con los demás hilos del mismo proceso la sección de código, los datos globales, el *heap* y los recursos del sistema (Silberschatz, Galvin y Gagne, 2018).

Compartir memoria vuelve a los hilos baratos de crear y rápidos para comunicarse, pero también es el origen de los problemas de este proyecto: dos hilos pueden leer y modificar la misma variable al mismo tiempo. Se distinguen los hilos **a nivel de usuario** (gestionados por una biblioteca, invisibles para el kernel) y los hilos **a nivel de kernel** (planificados individualmente por el sistema operativo). Solo estos últimos pueden ejecutarse en paralelo en varios núcleos y bloquearse de forma independiente en una llamada al sistema.

3.2 Hilos en Linux: modelo 1:1, NPTL y /proc

En Linux tanto procesos como hilos son **tareas** (*task_struct*) creadas con la llamada al sistema `clone()`. Un hilo es una tarea que comparte el espacio de memoria, los descriptores de archivo y los manejadores de señales de su creador (banderas `CLONE_VM`, `CLONE_FILES`, `CLONE_SIGHAND`, `CLONE_THREAD`). La biblioteca **NPTL** (*Native POSIX Thread Library*), incluida en glibc, implementa `pthread_create()` con un **modelo 1:1**: cada hilo POSIX corresponde exactamente a una tarea planificada por el kernel (Drepper y Molnar, 2005).

Cada hilo tiene un **TID** (identificador de hilo) distinto; todos los hilos de un proceso comparten el mismo **PID** (el `TGID`, *thread group id*). Las herramientas del sistema llaman *LWP* (*light-weight process*) al TID. Esto permite observar los hilos desde fuera del programa:

- `ps -eLf` muestra una fila por hilo; las columnas **LWP** (TID) y **NLWP** (número de hilos del proceso) prueban que existen varias tareas del kernel.
- `/proc/[pid]/task/` contiene un subdirectorío por hilo con `stat` (estado del planificador: **R** ejecutándose, **S** dormido interrumpible, **D** espera no interrumpible), `wchan` (función del kernel donde duerme), `syscall` (número de la llamada al sistema en curso) y `comm` (nombre del hilo).
- `htop -p [pid]` (tecla H) y `top -H` listan los hilos con su consumo de CPU; `strace -f` permite ver sus llamadas al sistema.

3.3 Hilos en Python: threading y el GIL

En CPython cada `threading.Thread` se crea con `pthread_create()`, por lo que en Linux es un **hilo NPTL real** con TID propio (`threading.get_native_id()`). El intérprete tiene un **cerrojo global** (GIL) que permite ejecutar *bytecode* de Python a un solo hilo a la vez; sin embargo, el GIL **no hace atómicas las operaciones compuestas**: el intérprete solicita un cambio de hilo cada `sys.getswitchinterval()` (5 ms por defecto) y libera el GIL durante cualquier llamada bloqueante al sistema (lectura o escritura de archivos, `fsync`, espera en un semáforo). Por eso

una secuencia «leer contador → verificar → escribir contador» puede ser interrumpida entre sus pasos y producir una condición de carrera, exactamente igual que en C con pthreads (Python Software Foundation, s. f.).

Los objetos de sincronización del módulo `threading` (`Lock`, `RLock`, `Semaphore`, `Condition`, `Event`, `Barrier`) están contruidos sobre las primitivas de la plataforma; en Linux terminan en la llamada al sistema `futex`, de modo que un hilo bloqueado en `sem.acquire()` aparece en el kernel en estado `S` con `wchan = futex_do_wait` y `syscall = 202` (`futex` en x86-64).

3.4 Concurrencia, sección crítica y condición de carrera

Una **condición de carrera** ocurre cuando varios hilos acceden a datos compartidos, al menos uno los modifica, y el resultado final depende del orden particular en que se intercalan sus instrucciones. El fragmento de código que accede a esos datos es la **sección crítica**. Una solución correcta al problema de la sección crítica debe cumplir tres requisitos (Silberschatz et al., 2018):

- **Exclusión mutua:** si un hilo está en su sección crítica, ningún otro puede estar en la suya.
- **Progreso:** si nadie está en la sección crítica, la decisión de quién entra no puede posponerse indefinidamente.
- **Espera acotada:** existe un límite al número de veces que otros entran antes de que un hilo que lo solicitó lo consiga.

El patrón más común es **verificar-y-actuar** (*check-then-act*): «si quedan copias, prestar una». Si dos hilos verifican antes de que cualquiera actualice, ambos creen que hay copia y el contador sufre una **actualización perdida** (*lost update*). En una biblioteca esto se traduce en prestar una copia que no existe (sobre-préstamo).

3.5 Mecanismos de sincronización

Mutex (lock)

Un *mutex* es un cerrojo binario con dueño: `acquire()` lo toma o bloquea al hilo si está ocupado; `release()` lo libera. Garantiza exclusión mutua sobre una sección crítica corta. En el proyecto, un `threading.Lock` protege únicamente el diccionario de estado interno (quién retiene qué copia y quién espera qué recurso), necesario para construir el RAG de forma consistente.

Semáforos

Dijkstra (1965) definió el **semáforo** como una variable entera `S` accesible solo mediante dos operaciones atómicas: **P(S)** o *wait*, que decrementa `S` y bloquea al hilo si el valor resultante no permite continuar, y **V(S)** o *signal*, que incrementa `S` y despierta a un hilo en espera. Un **semáforo binario** (0/1) equivale a un mutex; un **semáforo contador** inicializado en `N` modela un recurso con **N instancias idénticas**, por ejemplo `N` copias de un libro. En Python, `threading.Semaphore(n)` implementa esta semántica y `acquire(blocking=False)` realiza la verificación y el decremento en un solo paso atómico, devolviendo `False` si no hay instancias.

Monitores y variables de condición

Un **monitor** (Hoare, 1974) encapsula datos compartidos y los procedimientos que los manipulan, garantizando que solo un hilo ejecute dentro del monitor a la vez; las **variables de condición**

(`wait/notify`) permiten esperar a que se cumpla un predicado. En Python se obtienen combinando un `Lock` con `threading.Condition`; el proyecto las usa para los puntos de control del modo explicación.

Futex: la base en Linux

El **futex** (*fast userspace mutex*) combina una palabra entera en memoria compartida con la llamada al sistema `futex()`. Cuando no hay contención, el cerrojo se toma con una instrucción atómica sin entrar al kernel; cuando hay contención el hilo ejecuta `futex(FUTEX_WAIT)` y el kernel lo duerme hasta que otro hilo haga `futex(FUTEX_WAKE)` (Franke, Russell y Kirkwood, 2002; Drepper, 2011). Mutexes, semáforos POSIX y variables de condición de glibc están contruidos sobre futex: por eso la evidencia de un hilo bloqueado en el proyecto es `wchan = futex_do_wait` y no un ciclo de espera activa.

Tabla 1. Comparación de los mecanismos de sincronización usados.

Mecanismo	Semántica	Uso en BiblioSync
<code>threading.Semaphore(n)</code>	Contador de n instancias; P bloquea en 0, V despierta	Un semáforo por título (n = copias) y uno para las 4 estaciones de préstamo
<code>acquire(blocking=False)</code>	Verificar y decrementar atómico	Reservas en línea de la prueba de carrera (modo seguro)
<code>threading.Lock</code>	Exclusión mutua binaria con dueño	Protege el estado interno del que se construye el RAG
<code>threading.Condition</code>	Monitor: espera hasta que se cumpla un predicado	Puntos de control del «Modo Explicación»
<code>threading.Event / Barrier</code>	Señal de un solo uso / punto de encuentro de N hilos	Señal de aborto de la víctima / Llegada simultánea en hora pico
<code>queue.Queue</code>	Cola productor–consumidor segura para hilos	Cola de peticiones entrantes del generador

3.6 Transacciones, bloqueo en dos fases y bitácora

Una **transacción** es una secuencia de operaciones que debe ejecutarse de forma **atómica**: o se aplican todas o ninguna. El protocolo de **bloqueo en dos fases** (2PL) de Eswaran, Gray, Lorie y Traiger (1976) divide cada transacción en una **fase de crecimiento**, en la que solo adquiere bloqueos, y una **fase de decrecimiento**, en la que solo los libera; así se garantiza la serializabilidad. El 2PL **no evita** los interbloqueos: dos transacciones que retienen un bloqueo y piden el de la otra quedan en espera circular, que es precisamente lo que se inyecta en el Entregable 2.

Para poder deshacer una transacción abortada se usa una **bitácora de escritura anticipada** (WAL, *write-ahead log*): cada paso (BEGIN, ADQUIRIDO, COMMIT, ABORT, ROLLBACK) se escribe en disco **antes** de continuar. Con ella el sistema sabe exactamente qué recursos debe devolver durante el **rollback**.

3.7 Interbloqueos y condiciones de Coffman

Un conjunto de hilos está en **interbloqueo** cuando cada hilo del conjunto espera un evento que solo puede provocar otro hilo del mismo conjunto (Tanenbaum y Bos, 2015). Coffman, Elphick y Shoshani (1971) demostraron que un interbloqueo solo puede ocurrir si se cumplen **simultáneamente** cuatro condiciones:

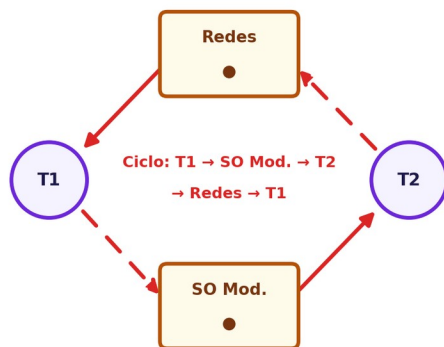
1. **Exclusión mutua:** al menos un recurso no es compartible (una copia física solo puede estar prestada a un usuario).
2. **Retención y espera:** un hilo retiene al menos un recurso mientras espera otros.
3. **No apropiación:** los recursos no se pueden quitar a la fuerza; solo los libera voluntariamente quien los tiene.
4. **Espera circular:** existe una cadena $T1 \rightarrow T2 \rightarrow \dots \rightarrow Tn \rightarrow T1$ en la que cada hilo espera un recurso retenido por el siguiente.

3.8 Grafo de Asignación de Recursos (RAG)

Holt (1972) propuso describir el estado de asignación con un grafo dirigido cuyos vértices son hilos (círculos) y tipos de recurso (rectángulos con un punto por instancia). Una **arista de solicitud** $T_i \rightarrow R_j$ indica que el hilo espera una instancia de R_j ; una **arista de asignación** $R_j \rightarrow T_i$ indica que una instancia de R_j está asignada a T_i . Sus propiedades fundamentales son:

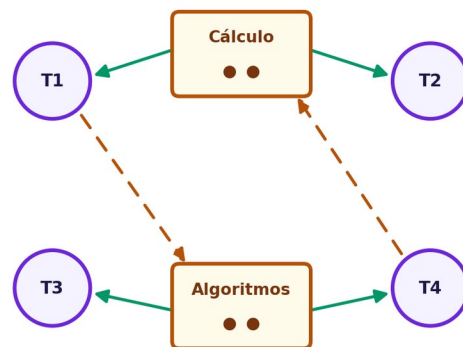
- Si el grafo **no tiene ciclos**, no hay interbloqueo.
- Si hay un ciclo y **cada recurso tiene una sola instancia**, hay interbloqueo (condición necesaria y suficiente).
- Si los recursos tienen **varias instancias**, un ciclo es condición necesaria pero **no suficiente**: un hilo fuera del ciclo puede liberar una instancia y romperlo (Figura 1b).

a) Una instancia por recurso: ciclo $\Rightarrow$ interbloqueo



Cada recurso tiene 1 copia: nadie fuera del ciclo puede liberar nada, por lo tanto el ciclo ES un interbloqueo

b) Varias instancias: ciclo sin interbloqueo



Ciclo $T1 \rightarrow \text{Algoritmos} \rightarrow T4 \rightarrow \text{Cálculo} \rightarrow T1$, pero T2 y T3 no esperan: al terminar liberan una copia y el ciclo se rompe (no hay interbloqueo)

— Asignación Recurso $\rightarrow$ Hilo — Solicitudo Hilo $\rightarrow$ Recurso — Arista del ciclo • Instancia (copia)

Figura 1. Grafo de asignación de recursos: (a) ciclo con recursos de una instancia, que es un interbloqueo; (b) ciclo con recursos de dos instancias, que no lo es. Elaboración propia con los títulos del catálogo del proyecto.

Cuando todos los recursos tienen una instancia se puede simplificar a un **grafo de espera** (*wait-for graph*) eliminando los nodos de recurso. Como en BiblioSync algunos títulos tienen 2 o 3 copias, el proyecto conserva el RAG completo y complementa la búsqueda de ciclos con el algoritmo de detección para múltiples instancias (sección 3.10).

3.9 Estrategias para manejar interbloqueos

Tabla 2. Estrategias clásicas de manejo de interbloqueos.

Estrategia	Idea	Relación con el proyecto
Prevención	Negar al menos una condición de Coffman. La más práctica es un orden total de recursos: todos piden en orden creciente, lo que hace imposible la espera circular (Havender, 1968).	Modo normal: cada transacción pide sus libros en orden ascendente de ISBN.
Evasión	Conceder una solicitud solo si el sistema queda en un estado seguro (algoritmo del banquero de Dijkstra). Requiere conocer por adelantado las necesidades máximas.	No se usa: los usuarios no declaran de antemano todo lo que pedirán.
Detección y recuperación	Permitir el interbloqueo, detectarlo periódicamente y recuperarse terminando o expropiando.	Modo caótico + T_Watchdog (DFS cada 0.5 s) + rollback de una víctima.
Ignorarlo («avestruz»)	Asumir que es raro y reiniciar si ocurre. Es lo que hacen Linux y Windows para la mayoría de recursos de usuario.	Se descarta: el objetivo es demostrar detección y recuperación.

3.10 Algoritmos de detección: DFS, Tarjan y Work/Finish

La **búsqueda en profundidad** (DFS) con tres colores detecta ciclos en un grafo dirigido en tiempo $O(V + E)$: cada nodo empieza **blanco**; al visitarlo se pinta **gris** (está en la pila de recursión) y al terminar sus sucesores se pinta **negro**. Si durante el recorrido se encuentra una arista hacia un nodo **gris** (*arista de retroceso*), existe un ciclo, que se reconstruye siguiendo los padres desde el nodo actual. El algoritmo de **Tarjan** (1972) usa la misma DFS para obtener las componentes fuertemente conexas; toda componente con más de un nodo contiene un ciclo.

Para recursos con varias instancias, Silberschatz et al. (2018) describen el **algoritmo de detección** con los vectores *Disponible* y las matrices *Asignación* y *Solicitud*:

```

Work    = Disponible
Finish[i] = (Asignación[i] == 0)      # quien no retiene nada no bloquea a nadie
repetir:
    buscar i con Finish[i] == False y Solicitud[i] <= Work
    si existe: Work += Asignación[i]; Finish[i] = True  # i termina y libera
hasta que no exista tal i
Los hilos con Finish[i] == False están INTERBLOQUEADOS.  # costo  $O(m \cdot n^2)$ 

```

El hilo guardián del proyecto aplica ambos: la DFS encuentra y dibuja el ciclo, y el algoritmo Work/Finish confirma que los hilos del ciclo realmente no pueden avanzar (evitando falsos positivos con los títulos de varias copias).

3.11 Recuperación: víctima, rollback e inanición

Una vez detectado el interbloqueo se puede (a) terminar a todos los hilos del ciclo, (b) terminarlos uno a uno hasta romperlo, o (c) expropiar recursos a una **víctima** y hacer **rollback** de su transacción. El criterio de selección busca el menor costo: prioridad, trabajo ya realizado, recursos retenidos o antigüedad. El riesgo es la **inanición**: que siempre se elija a la misma víctima; se mitiga incluyendo el número de rollbacks en el costo o limitando los reintentos (Silberschatz et al., 2018; Stallings, 2018). BiblioSync elige al hilo que entró **más recientemente** al ciclo (el que menos trabajo pierde), revierte con la bitácora y reintenta su solicitud al final, con el sistema libre y en orden de ISBN, de modo que la víctima siempre termina.

3.12 Comunicación en tiempo real con el navegador

El protocolo **WebSocket** (Fette y Melnikov, 2011) abre un canal bidireccional y persistente sobre TCP entre el navegador y el servidor, lo que permite enviar el estado del motor cada 250 ms sin que el cliente lo solicite. **Flask-SocketIO** integra Socket.IO con Flask; configurado con `async_mode="threading"` usa hilos del sistema operativo (y no *green threads* de eventlet o gevent, que no serían hilos del kernel). En el navegador, la biblioteca **vis-network** dibuja el RAG como un grafo interactivo.

4. Entregable 0 — Propuesta del proyecto

Origen de esta sección

Resume y formaliza la *Hoja de Trabajo — Propuesta de Proyecto 2* elaborada en clase (2 horas), respondiendo cada punto que la guía exige para el Entregable 0.

4.1 Escenario elegido y por qué genera contención real

Sistema de préstamo de libros de una biblioteca universitaria con varias sucursales conectadas al mismo catálogo. Cada título tiene un número fijo de copias físicas y los usuarios pueden pedir **préstamos combinados** (dos o más libros en una misma transacción).

- La contención es real porque los títulos más solicitados (por ejemplo, en semana de parciales) tienen **una sola copia**, y varias sucursales atienden al mismo tiempo sobre el mismo catálogo.
- Si dos solicitudes piden los mismos dos libros en **orden opuesto** ($A \rightarrow B$ y $B \rightarrow A$), compiten por el mismo par de copias limitadas: el problema no es trivial ni está resuelto por un sistema existente.

Tabla 3. Catálogo usado por el motor (valor inicial de cada semáforo).

Título (nombre corto)	Autor	Copias	Rol en las pruebas
Sistemas Operativos Modernos (SO Modernos)	A. S. Tanenbaum	1	Título popular: genera espera circular
Fundamentos de Sistemas Operativos (Silberschatz)	A. Silberschatz	2	Recurso multi-instancia
Redes de Computadoras (Redes)	J. Kurose	1	Título popular
Fundamentos de Bases de Datos (Bases de Datos)	H. Korth	1	Título popular
Cálculo de una Variable (Cálculo)	J. Stewart	3	Prueba de condición de carrera
Introducción a los Algoritmos (Algoritmos)	T. Cormen	2	Recurso multi-instancia
Estaciones de préstamo	—	4	Recurso contable compartido por las sucursales

4.2 Lenguaje, entorno y mecanismo de sincronización

- **Lenguaje y entorno:** Python 3 en **WSL2 (Ubuntu)**, módulo `threading`. Los hilos son verificables con `ps -eLf` y `/proc/[pid]/task`, igual que en el laboratorio de hilos.
- **Mecanismo elegido:** **semáforos** (`threading.Semaphore`), uno por título, porque modelan de forma natural un **recurso contable** (N copias) y no una exclusión binaria. Otro semáforo contador limita las estaciones de préstamo.

- **Lock adicional:** un `threading.Lock` protege solo las actualizaciones del diccionario de estado interno (qué hilo tiene qué copia y quién espera qué), necesario para construir el RAG de forma consistente.

4.3 Recursos compartidos e hilos que compiten

- **Recursos:** copias físicas de cada título (contador por ISBN) y un número limitado de estaciones/cajeros de préstamo.
- **Hilos:** cada solicitud de préstamo corre como un hilo real (`T_Usuario01`, `T_Usuario02`, ...) que representa a un usuario pidiendo uno o varios libros en la misma transacción. Además existen `T_Generador` (lee la cola de peticiones), `T_Watchdog` (hilo guardián) y `T_Broadcast` (envía el estado al navegador).
- **Competencia:** dos o más hilos piden copias del mismo título a la vez, o una transacción combinada necesita dos títulos simultáneamente.

4.4 Cómo provocaremos y comprobaremos la condición de carrera

Provocación. Sin protección, la reserva sigue el patrón verificar-y-actuar: el hilo lee `disponibles`, si es mayor que cero **registra la reserva en la bitácora** (escritura real en disco con `os.write` + `os.fsync`, durante la cual el hilo cede la CPU) y después escribe `disponibles - 1`. No se usa `time.sleep()`: la ventana entre leer y escribir la abre una operación de E/S genuina. Se lanzan 12 reservas en línea simultáneas (una `threading.Barrier` las hace llegar al mismo tiempo, como en la apertura de las sucursales) sobre «Cálculo», que tiene 3 copias.

Tabla 4. Intercalado que produce el sobre-préstamo (ejemplo con dos sucursales y 1 copia).

Tiempo	Hilo A — Sucursal Central	Hilo B — Sucursal Chimaltenango	disponibles
t1	lee disponibles = 1	—	1
t2	escribe en la bitácora (E/S: cede la CPU)	lee disponibles = 1	1
t3	escribe disponibles = 0	escribe en la bitácora	0
t4	préstamo confirmado	escribe disponibles = 0 (valor obsoleto)	0
Resultado	2 préstamos de un título con 1 copia (sobre-préstamo) y una actualización perdida: el contador dice 0 pero hay 2 copias «prestadas».		

Comprobación. El motor audita cada préstamo registrado; se verifica el invariante `disponibles + prestadas = copias` y `prestadas ≤ copias`. En el panel el título se marca en rojo con copias «fantasma» y la tarjeta de integridad pasa a **VIOLADA**. El script `scripts/prueba_carga.py` repite la prueba con 100 hilos × 200 operaciones y cuenta las violaciones.

Corrección. Con la protección activada, la reserva usa `sem.acquire(blocking=False)`, que verifica y decrementa en una sola operación atómica: exactamente 3 reservas tienen éxito y 9 son rechazadas, sin importar el intercalado.

4.5 Diseño inicial de la espera circular (modo caótico)

- **Operación normal:** toda solicitud combinada adquiere los títulos en el **mismo orden fijo (ascendente por ISBN)**, lo que niega la espera circular y hace imposible el ciclo.
- **Modo caótico:** el generador lee el orden de los libros **directamente de la cola de peticiones entrantes** (el orden en que cada usuario los pidió), sin reordenarlos, lo que permite órdenes cruzados.
- La espera circular surge de la **lógica real de asignación** (quién llegó primero a cada semáforo según el planificador de Linux), no de un `sleep()` colocado a propósito. Si una oleada no llega a cruzarse, el generador toma la siguiente oleada de la cola.

Tabla 5. Ejemplo de cómo se forma el ciclo (caso real capturado en la ejecución de prueba).

Hilo	Orden en la cola	Obtiene primero	Pide después	Retenido por
T_Usuario11	SO Modernos → Bases de Datos	SO Modernos	Bases de Datos	T_Usuario12
T_Usuario12	Bases de Datos → Redes	Bases de Datos	Redes	T_Usuario09
T_Usuario09	Redes → SO Modernos	Redes	SO Modernos	T_Usuario11
Ciclo	T_Usuario11 → «Bases de Datos» → T_Usuario12 → «Redes» → T_Usuario09 → «SO Modernos» → T_Usuario11			

4.6 Estrategia de detección y recuperación propuesta

- **Estructura de datos del RAG:** diccionario de adyacencia (grafo dirigido) con nodos = hilos y recursos (títulos con sus copias). Las aristas de asignación (recurso → hilo) y de espera (hilo → recurso) se actualizan en cada `acquire()/release()` mediante un envoltorio alrededor del semáforo (`SemaforoRastreado`).
- **Detección:** un hilo guardián ejecuta **DFS** periódicamente (cada 0.5 s) sobre el grafo para encontrar ciclos.
- **Recuperación:** al detectar el ciclo, el guardián aborta (rollback) la transacción del hilo que entró **más recientemente** al ciclo, liberando los semáforos ya adquiridos. El resto de los hilos continúa sin reiniciar el sistema.

4.7 Propuesta de frontend

Interfaz web con **Flask + Flask-SocketIO**: panel de hilos activos, tabla de copias disponibles/prestadas por título y el grafo RAG dibujado con **vis.js**, resaltando en rojo el ciclo si hay interbloqueo, más un botón para activar el modo caótico. Todo se alimenta del estado real del backend por WebSocket en cada evento de `acquire/release/detección`, no de datos de ejemplo. El boceto de la hoja de trabajo se convirtió en el panel de la sección 5.6.

4.8 Cronograma de trabajo hasta la entrega final

Tabla 6. Cronograma propuesto (fechas ajustables al calendario oficial del curso).

Semana	Fechas	Actividades	Entregable	Responsable(s)
1	28 sep – 4 oct	Entrega de la propuesta. Repositorio, catálogo, semáforos por título, bitácora WAL, prueba de condición de carrera.	Entregable 0	
2	5 – 11 oct	Motor concurrente completo (2PL), estaciones de préstamo, prueba de carga, capturas de ps -eL f y /proc.	Entregable 1	
3	12 – 18 oct	Modo caótico, construcción del RAG, hilo guardián con DFS + Work/Finish, rollback y reintento.	Entregable 2	
4	19 – 25 oct	Panel Flask-SocketIO + vis-network, monitor de /proc, modo explicación con pausas.	Avance E2/E3	
5	26 oct – 1 nov	README, página de la Wiki, comentarios del código, ensayo y demostración en vivo.	Entregable 3	

5. Diseño e implementación

5.1 Arquitectura

Todo el sistema corre en **un solo proceso** Linux (`python3 app.py`) con varios hilos NPTL. La Figura 2 muestra quién crea a quién y por dónde fluye la información. El navegador no contiene datos propios: recibe cada 250 ms una copia del estado del motor y de `/proc`.

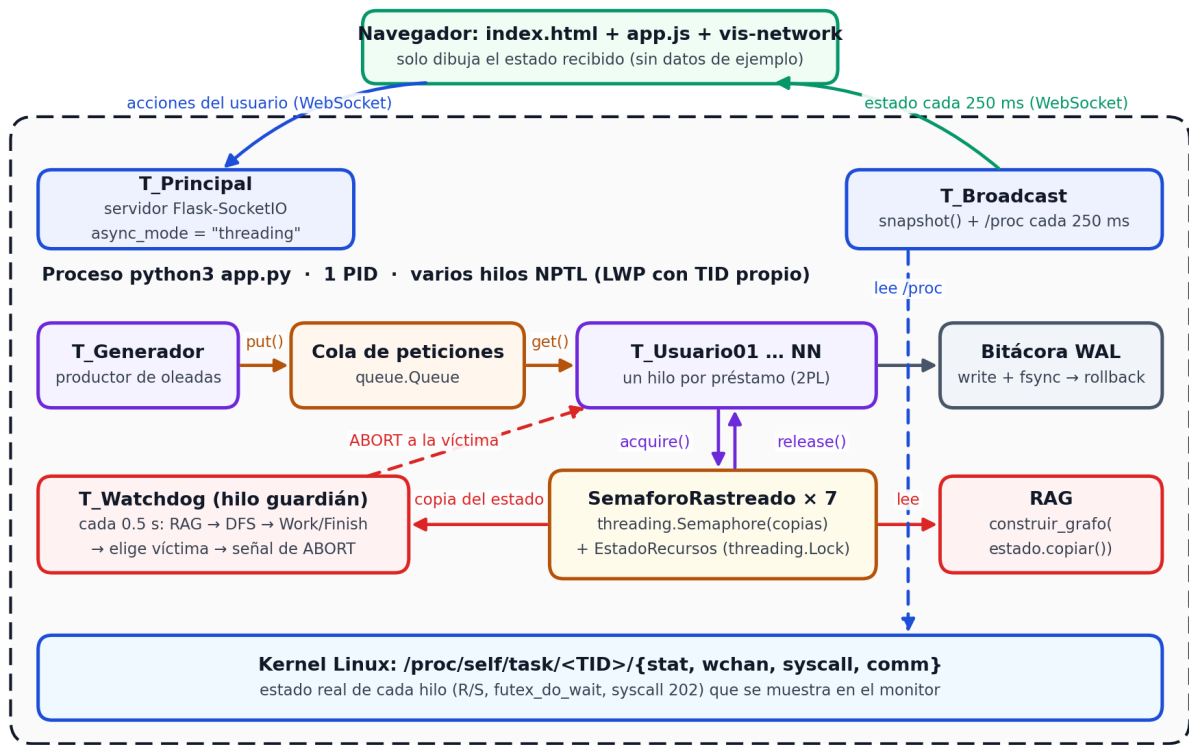


Figura 2. Arquitectura de BiblioSync: hilos del proceso, recursos sincronizados, hilo guardián y canal WebSocket con el navegador.

5.2 Módulos del repositorio

Tabla 7. Responsabilidad de cada módulo.

Archivo	Responsabilidad
<code>app.py</code>	Servidor Flask-SocketIO en modo <i>threading</i> , hilo <code>T_Broadcast</code> , API <code>/api/estado</code> y eventos del panel.
<code>motor/catalogo.py</code>	Títulos, número de copias (valor inicial de cada semáforo), estaciones e ISBN (orden global).
<code>motor/recursos.py</code>	<code>SemaforoRastreado</code> (semáforo real + aristas del RAG), <code>EstadoRecursos</code> (Lock) y la versión sin protección.
<code>motor/transacciones.py</code>	<code>HiloPrestamo</code> (2PL, bitácora, rollback) y <code>HiloReservaEnLinea</code> (prueba de carrera).

Archivo	Responsabilidad
motor/generador.py	Cola de peticiones (<code>queue.Queue</code>) y <code>T_Generador</code> (productor–consumidor, oleadas, reintentos).
motor/rag.py	Construcción del RAG, DFS con colores y algoritmo Work/Finish.
motor/watchdog.py	<code>T_Watchdog</code> : detección periódica, confirmación y recuperación por víctima.
motor/kernel.py	Lectura de <code>/proc/self/task/<TID>/{stat, wchan, syscall, comm}</code> y nombre del hilo con <code>prctl(PR_SET_NAME)</code> .
motor/bitacora.py	Bitácora WAL (<code>os.write + os.fsync</code>).
motor/pasos.py, explicaciones.py	Modo explicación (puntos de control con <code>Condition</code>) y textos dinámicos.
scripts/evidencia.sh	Captura de evidencias con herramientas del SO.
scripts/prueba_carga.py	Prueba de integridad bajo carga con y sin semáforos.

5.3 Ciclo de vida de una transacción de préstamo

1. `T_Generador` saca la solicitud de la cola y crea el hilo; una `threading.Barrier` hace que todas las solicitudes de la oleada lleguen a la vez (hora pico).
2. El hilo se nombra con `prctl` (visible en `ps`, `htop` y `/proc/[pid]/task/[tid]/comm`) y toma una **estación de préstamo** (semáforo de 4).
3. Escribe **BEGIN** en la bitácora y entra en la **fase de crecimiento**: por cada libro (en orden de ISBN o en el orden de la cola) registra la arista de solicitud, ejecuta `sem.acquire()` —si no hay copia, duerme en `futex`— y, al obtenerla, convierte la solicitud en arista de asignación y escribe **ADQUIRIDO** con `fsync`.
4. Con todos los libros escribe **COMMIT**, libera la estación y mantiene el préstamo durante un periodo simulado de lectura (fuera de toda adquisición).
5. **Fase de decrecimiento**: devuelve las copias en orden inverso con `release()` (**DEVUELTO**), lo que despierta a quien esperaba.
6. Si recibe la señal de aborto del guardián mientras espera, lanza `TransaccionAbortada`, escribe **ABORT** y ejecuta **ROLLBACK**: libera en orden inverso todo lo que tomó.

5.4 El RAG se construye desde el estado real

`SemaforoRastreado` envuelve al `threading.Semaphore` y, bajo el `Lock` del estado, registra la solicitud antes de bloquear y la asignación después de obtener la copia. El RAG se obtiene de una copia consistente de esas mismas estructuras; no existe ningún grafo dibujado a mano.

```
def adquirir(self, hilo):
    # recursos.py (resumido)
    with est.lock: est.esperas[hilo.name] = (self.rid, ahora) # arista Hilo -> Recurso
    while not self._sem.acquire(timeout=0.25):
        # P(S) real: el hilo duerme en futex
        if hilo.abortar.is_set():
            # señal del watchdog (víctima)
            raise TransaccionAbortada(...)
```

```

with est.lock:
    # la solicitud se convierte en asignación
    del est.esperas[hilo.name]; r.disponibles -= 1; r.retenedores[hilo.name] += 1

def construir_grafo(snap):
    # rag.py
    for rid, ret in snap['asignacion'].items(): ady['R:'+rid] += ['T:'+h for h in ret]
    for h, (rid, t) in snap['esperas'].items(): ady['T:'+h].append('R:'+rid)

```

La espera con `acquire(timeout=0.25)` en un bucle no cambia la semántica del semáforo (el hilo sigue bloqueado en el kernel mientras no haya copia); solo le permite atender la señal de aborto del guardián, porque un hilo no puede ser «matado» desde fuera en Python.

5.5 Hilo guardián: detección y recuperación

```

cada 0.5 s:
    snap = estado.copiar()
    grafo = construir_grafo(snap)
    ciclo = dfs_buscar_ciclo(grafo)
    bloq = detectar_interbloqueados(snap)
    si ciclo y hilos(ciclo) ⊆ bloq en 2 escaneos seguidos:
        declarar INTERBLOQUEO (log + panel + bitácora)
        si recuperación automática o botón «Resolver»:
            víctima = hilo del ciclo con la solicitud MÁS RECIENTE
            víctima.abortar.set()
            su solicitud vuelve a la cola (reintento con sistema libre y orden ISBN)

```

5.6 Frontend en tiempo real

El panel replica el diseño de referencia mostrado en clase, adaptado al escenario de la biblioteca: cabecera con kernel, PID y número de hilos del SO; botones de escenarios; interruptores de pausas, protección y recuperación; un panel de **pausa para explicación** con tres columnas (qué ocurrió en pantalla, concepto de sistemas operativos y evidencia en el kernel de Linux); métricas; y en la parte inferior la estantería de recursos, el **RAG dinámico** y el **monitor de `/proc/[PID]/task`** con el log del backend.

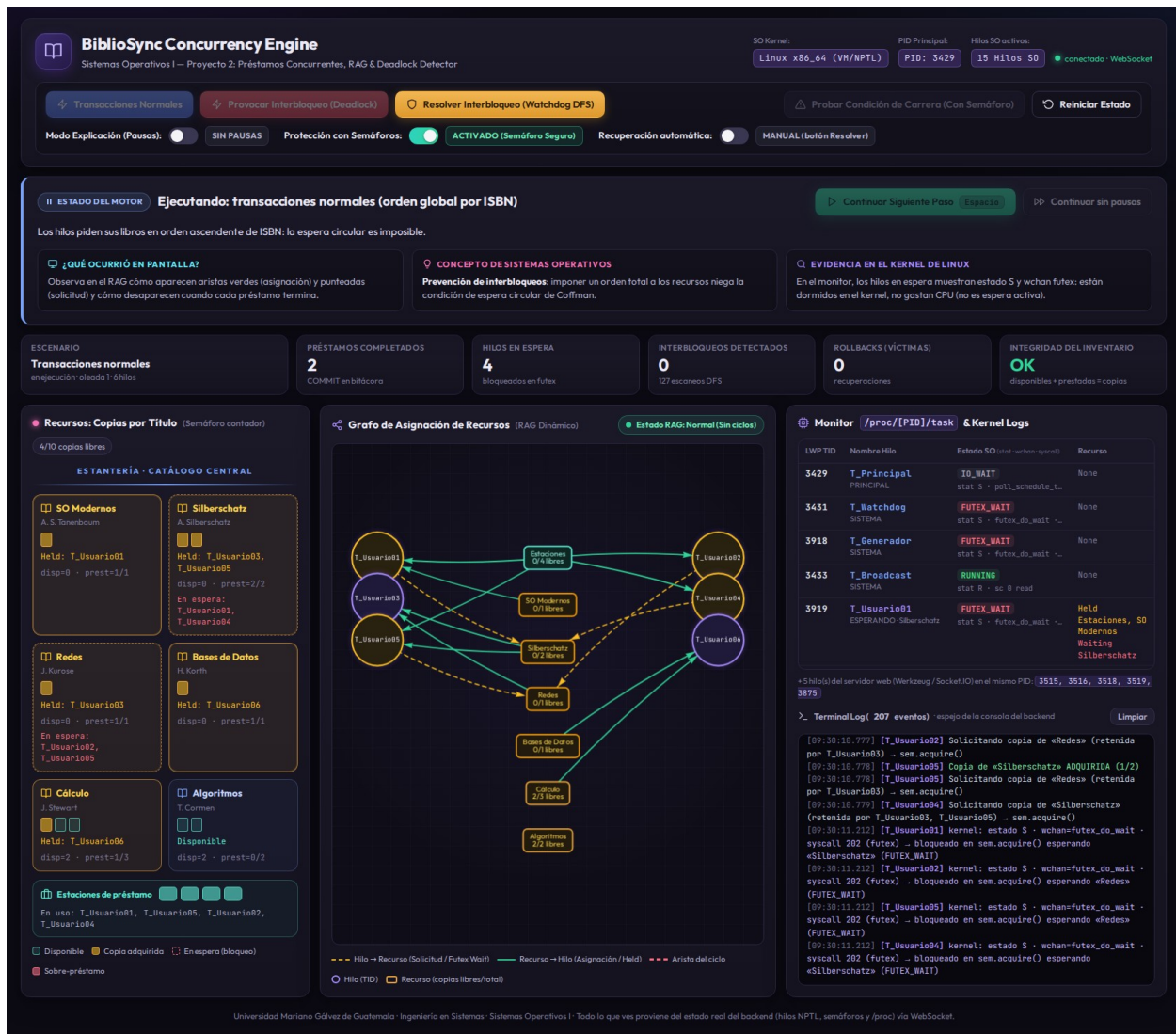


Figura 3. Panel de BiblioSync durante el escenario de transacciones normales: cuatro hilos duermen en futex esperando «Silberschatz» o «Redes» (pedidos en orden de ISBN); el RAG no tiene ciclos.

¿El «Modo Explicación» actúa el interbloqueo?

No. Las pausas son **puntos de control didácticos** (como los *breakpoints* de un depurador): el hilo que llega a uno espera en una *threading.Condition* hasta que el expositor presiona *Continuar*. No deciden el orden en que cada hilo pide sus libros ni quién gana un semáforo. En el modo caótico solo existe un punto de control (después del primer libro); luego los hilos corren libres y la siguiente pausa la provoca el propio interbloqueo detectado por el guardián.

Con «Continuar sin pausas» el motor corre sin ningún punto de control y los mismos fenómenos aparecen solos, lo que se verificó repetidamente en las pruebas (sección 7.4).

6. Entregable 1 — Motor concurrente e integridad bajo carga

6.1 Prueba de condición de carrera en el panel

Con **Protección con Semáforos: DESACTIVADO**, doce reservas en línea simultáneas de «Cálculo» (3 copias) leyeron el contador al mismo tiempo (`disponibles = 3`), escribieron $3 - 1 = 2$ y registraron **12 préstamos**: 9 copias fantasma y un contador final de 2 que no coincide con nada. La tarjeta de integridad del inventario pasa a VIOLADA.

Fase 1: Lectura concurrente del contador SIN protección

12 reservas desde varias sucursales leyeron el contador de «Cálculo» al mismo tiempo.

¿QUÉ OCURRIÓ EN PANTALLA?
Todos leyeron `disponibles = 3` y creen que hay copia. Todavía nadie escribió.

CONCEPTO DE SISTEMAS OPERATIVOS
Condición de carrera verificar-y-actuar: leer y escribir no son atómicos; entre ambas instrucciones el planificador ejecuta a otro hilo.

EVIDENCIA EN EL KERNEL DE LINUX
Sin semáforo no hay futex: los hilos no se bloquean entre sí. El GIL cambia de hilo cada `sys.getswitchinterval()` y en cada E/S (fsync de la bitácora).

Fase 2: Escritura con valor obsoleto → SOBRE-PRÉSTAMO

Se registraron 12 préstamos de un título con 3 copias. 9 copias fantasma.

¿QUÉ OCURRIÓ EN PANTALLA?
Cada hilo escribió `disponibles = leído - 1` (actualización perdida / lost update). Contador final: 2. Invariante violado: prestados (12) > copias (3).

CONCEPTO DE SISTEMAS OPERATIVOS
Sección crítica sin exclusión mutua. Solución: `sem_acquire(blocking=False)` verifica y decrementa en un solo paso atómico.

EVIDENCIA EN EL KERNEL DE LINUX
Activa 'Protección con Semáforos' y repite: exactamente tantos préstamos como copias; el resto rechazados.

Recursos: Copias por Título (Semáforo contador)

9/10 copias libres

ESTANTERÍA - CATÁLOGO CENTRAL

- SO Modernos** (A. S. Tanenbaum): Disponible, disp=1, prest=0/1
- Silberschatz** (A. Silberschatz): Disponible, disp=2, prest=0/2
- Redes** (J. Kurose): Disponible, disp=1, prest=0/1
- Bases de Datos** (H. Korth): Disponible, disp=1, prest=0/1
- Cálculo** (J. Stewart): Disponible, Held: T_Reserva11, T_Reserva06, T_Reserva02, T_Reserva09, T_Reserva04, T_Reserva12, T_Reserva08, T_Reserva10, T_Reserva05, T_Reserva03, T_Reserva01, disp=2, prest=12/3
- Algoritmos** (T. Cormen): Disponible, disp=2, prest=0/2

Estaciones de préstamo

Todas libres

Grafo de Asignación de Recursos (RAG Dinámico)

Estado RAG: Normal (Sin ciclos)

Monitor /proc/[PID]/task & Kernel Logs

LWP TID	Nombre Hilo	Estado SO	wait -wchan syscall	Recurso
3429	T_Principal PRINCIPAL	IDLE_WAIT	stat S · poll, schedule, t...	None
3431	T_Watchdog SISTEMA	FUTEX_WAIT	stat S · futex, do_wait, ...	None
3433	T_Broadcast SISTEMA	RUNNING	stat R · sc 0 read	None
3880	T_Reserva01 PRÉSTAMO ACTIVO: Cálculo	FUTEX_WAIT	stat S · futex, do_wait, ...	Held Cálculo
3881	T_Reserva02 PRÉSTAMO ACTIVO: Cálculo	FUTEX_WAIT	stat S · futex, do_wait, ...	Held Cálculo
3882	T_Reserva03	FUTEX_WAIT	stat S · futex, do_wait, ...	Held Cálculo

4 Hilos del servidor web (Werkzeug / Socket.IO) en el mismo PID: 3515, 3516, 3518, 3519, 3875

Terminal Log (160 eventos) - espejo de la consola del backend

```

escribió 2 - RESERVA registrada SIN protección
[09:30:05.021] [T_Reserva09] [Villa Nueva] leyó disponibles = 3 -
escribió 2 - RESERVA registrada SIN protección
[09:30:05.021] [T_Reserva04] [Antigua] leyó disponibles = 3 -
escribió 2 - RESERVA registrada SIN protección
[09:30:05.021] [T_Reserva12] [Zona 1] leyó disponibles = 3 - escribió
2 - RESERVA registrada SIN protección
[09:30:05.021] [T_Reserva08] [Osinatmanapo] leyó disponibles = 3 -
escribió 2 - RESERVA registrada SIN protección
[09:30:05.021] [T_Reserva10] [Antigua] leyó disponibles = 3 -
escribió 2 - RESERVA registrada SIN protección
[09:30:05.021] [T_Reserva05] [Máximo] leyó disponibles = 3 - escribió
2 - RESERVA registrada SIN protección
[09:30:05.021] [T_Reserva03] [Villa Nueva] leyó disponibles = 3 -
escribió 2 - RESERVA registrada SIN protección
[09:30:05.021] [T_Reserva01] [Central] leyó disponibles = 3 -
escribió 2 - RESERVA registrada SIN protección
[09:30:05.070] [MOTOR] Resultado: 12 préstamos registrados de 3
copias - disponibles = 2 - integridad VIOLADA
  
```

Figura 4. Prueba sin protección (Modo Explicación): lectura concurrente, escritura con valor obsoleto y resultado de 12 préstamos de 3 copias.

Con **Protección con Semáforos: ACTIVADO**, la misma prueba usa `acquire(blocking=False)`: exactamente 3 reservas obtienen copia, 9 son rechazadas y el inventario queda consistente.

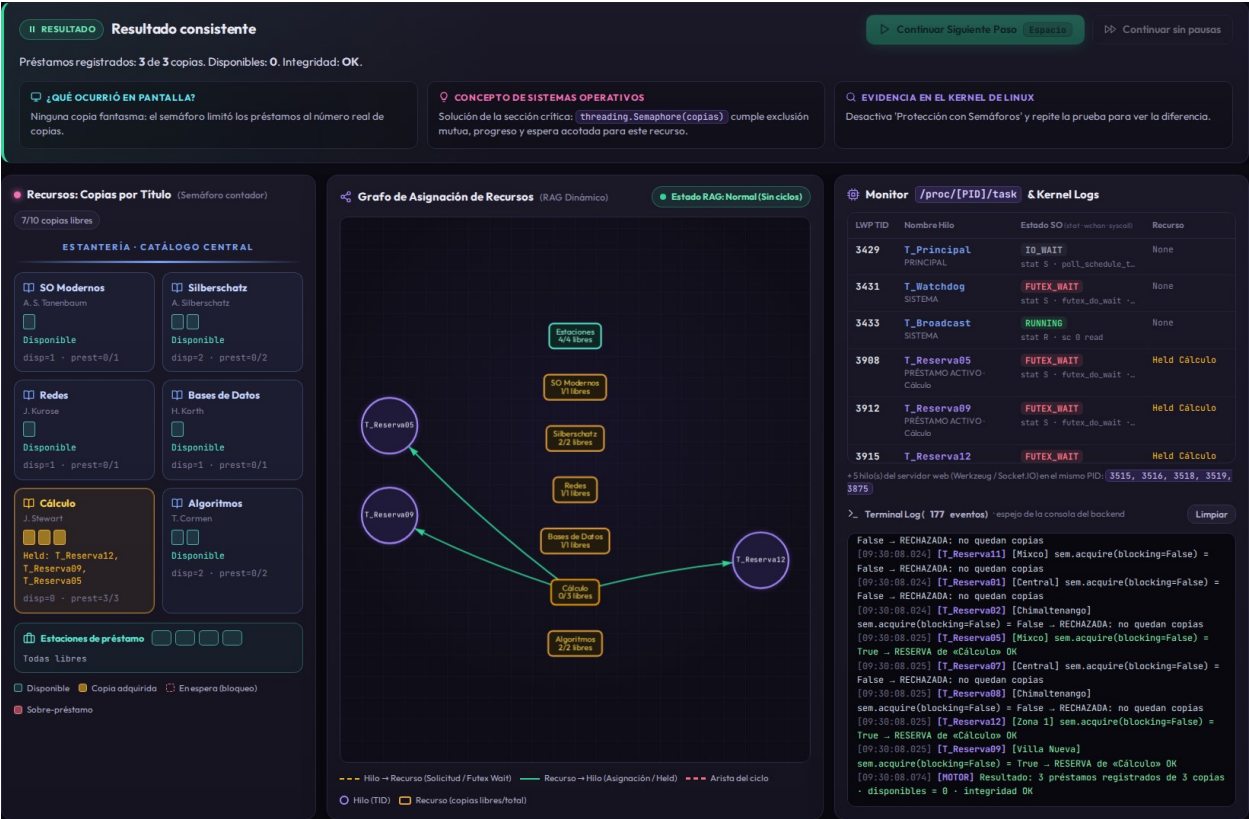


Figura 5. Prueba con semáforo: 3 préstamos de 3 copias, 9 rechazos, integridad OK.

6.2 Prueba de carga (100 hilos × 200 operaciones)

`scripts/prueba_carga.py` lanza 100 hilos reales (101 entradas en `/proc/self/task`) que realizan 200 préstamos/devoluciones cada uno sobre los seis títulos, primero sin protección y luego con semáforos, auditando en cada préstamo que no haya más préstamos simultáneos que copias.

Tabla 8. Resultados de la prueba de carga (ejecución del 26/09/2026).

Métrica	Sin protección	Con semáforos
Operaciones intentadas	20 000	20 000
Préstamos realizados	12 026	2 120
Violaciones «prestadas > copias» observadas	10 448	0
Títulos con contador final ≠ copias	6 de 6	0 de 6
Máximo simultáneo de «Redes» (1 copia)	12	1
Máximo simultáneo de «Cálculo» (3 copias)	12	3
Resultado	INTEGRIDAD VIOLADA	INTEGRIDAD OK

Sin protección hay más «préstamos» porque el sistema presta copias que no existen; con semáforos hay más rechazos porque el sistema respeta el número real de copias. En los seis

títulos el máximo de préstamos simultáneos con semáforos coincide exactamente con el número de copias.

7. Entregable 2 — Interbloqueo, RAG y recuperación

Se activó **Provocar Interbloqueo (Deadlock)** con el Modo Explicación para documentar el ciclo completo: operación → interbloqueo inyectado → detección → recuperación.

7.1 Fase 1: hora pico, cada hilo toma su primer libro

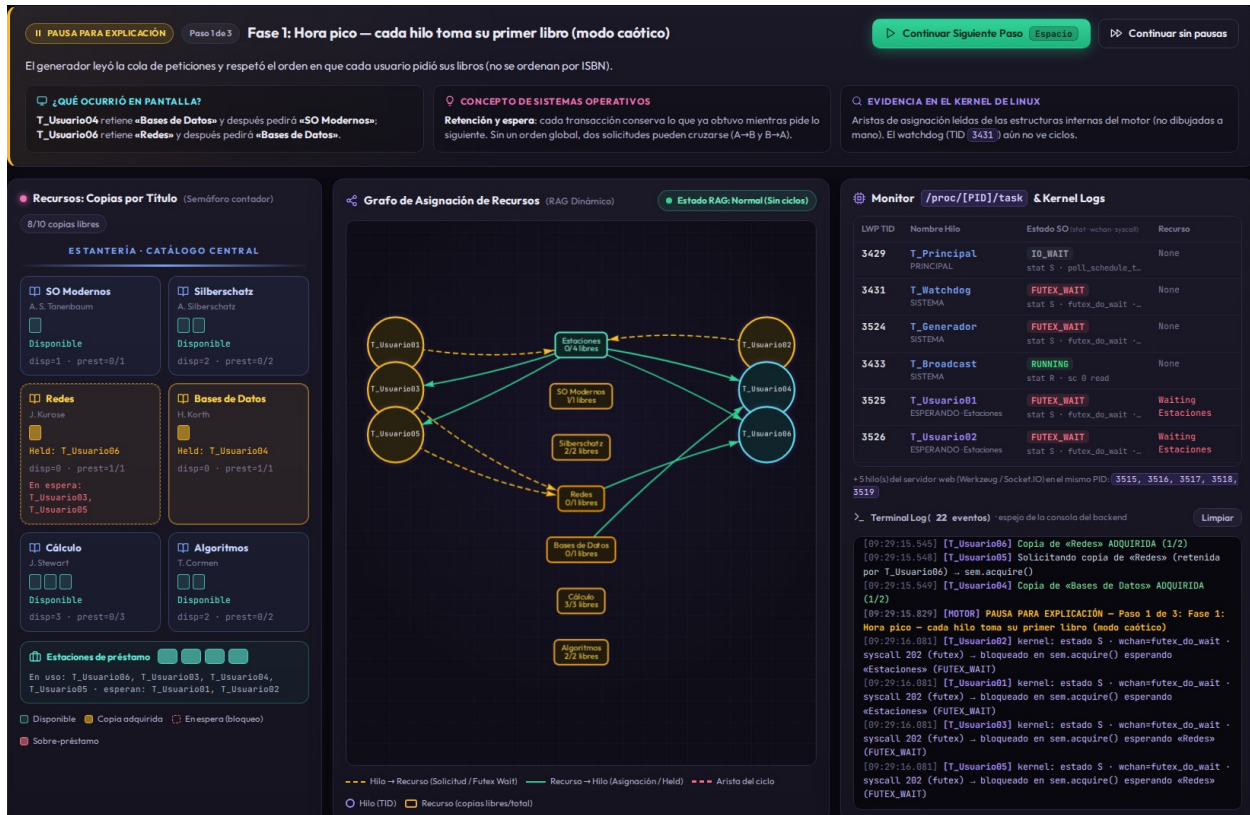


Figura 6. Fase 1 del modo caótico: el generador respetó el orden de la cola; cada hilo retiene su primer libro (aristas verdes) y los demás duermen en futex esperando estación o libro.

7.2 Fase 2: espera circular detectada por el watchdog

Al continuar, cada hilo pidió su segundo libro y quedó bloqueado. El hilo guardián encontró con DFS el ciclo **T_Usuario11** → «Bases de Datos» → **T_Usuario12** → «Redes» → **T_Usuario09** → «SO Modernos» → **T_Usuario11**, lo confirmó con Work/Finish en dos escaneos y lo resaltó en rojo. Los tres hilos del ciclo aparecen en el kernel en estado **S**, **wchan = futex_do_wait**, **syscall 202**.

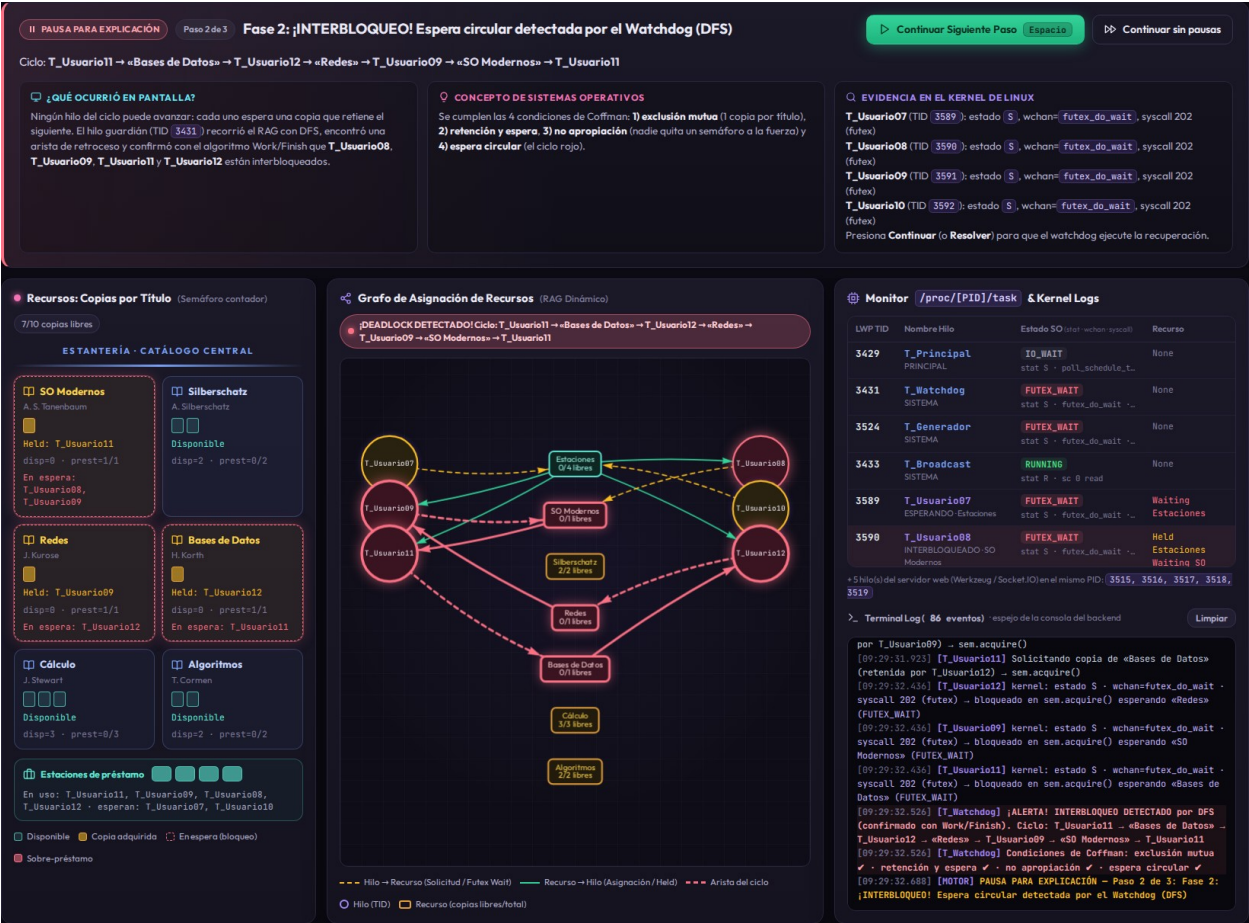


Figura 7. Fase 2: interbloqueo real entre tres hilos. El RAG, la estantería y el monitor de /proc muestran el mismo ciclo.

Tabla 9. Las cuatro condiciones de Coffman en BiblioSync.

Condición	Cómo se cumple en el modo caótico	Cómo se rompe
Exclusión mutua	Los títulos populares tienen 1 copia: <code>threading.Semaphore(1)</code> .	Inherente al recurso (una copia física no se comparte).
Retención y espera	El hilo conserva el primer libro mientras pide el segundo (fase de crecimiento del 2PL).	Se podría pedir todo de una vez; no se hace para reflejar el préstamo real.
No apropiación	Un semáforo solo lo libera quien lo tomó.	Recuperación: el guardián obliga a la víctima a abortar y liberar (rollback).
Espera circular	El orden de la cola cruza las solicitudes (A → B, B → C, C → A).	Prevención (modo normal): orden total por ISBN.

7.3 Fase 3: recuperación por rollback

Al presionar *Continuar* (equivalente a *Resolver Interbloqueo*), el guardián eligió como víctima al hilo cuya solicitud era la más reciente (**T_Usuario11**), le envió la señal de aborto y este ejecutó ROLLBACK liberando su libro y su estación. En esta ejecución, al liberarse esas copias se formó

de inmediato un **segundo ciclo** con otros hilos que esperaban, y el guardián lo resolvió igual (víctima **T_Usuario08**), lo que muestra que la detección es continua y no depende de un guion. Los demás hilos despertaron (FUTEX_WAKE) y completaron sus préstamos; las solicitudes de las víctimas se reintentaron al final con el sistema libre y en orden de ISBN. Resultado de las dos oleadas: 12 préstamos completados, 2 interbloqueos, 2 rollbacks e integridad OK.

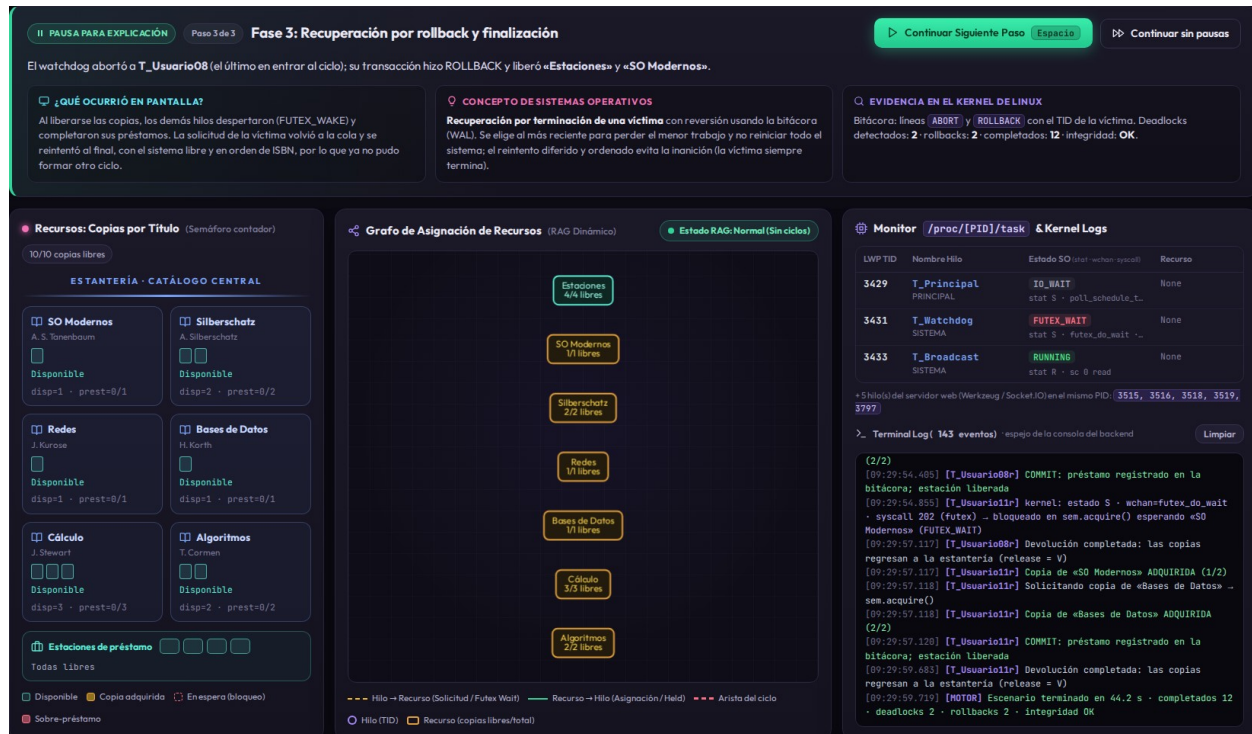


Figura 8. Fase 3: RAG sin ciclos, inventario completo y resumen de la recuperación.

7.4 Sin pausas: el interbloqueo aparece por sí solo

Con el Modo Explicación desactivado no existe ningún punto de control. En las pruebas, el modo caótico produjo interbloqueos espontáneos en la primera o segunda oleada; la Figura 9 muestra uno detectado sin pausas, a la espera del botón *Resolver* (recuperación manual).

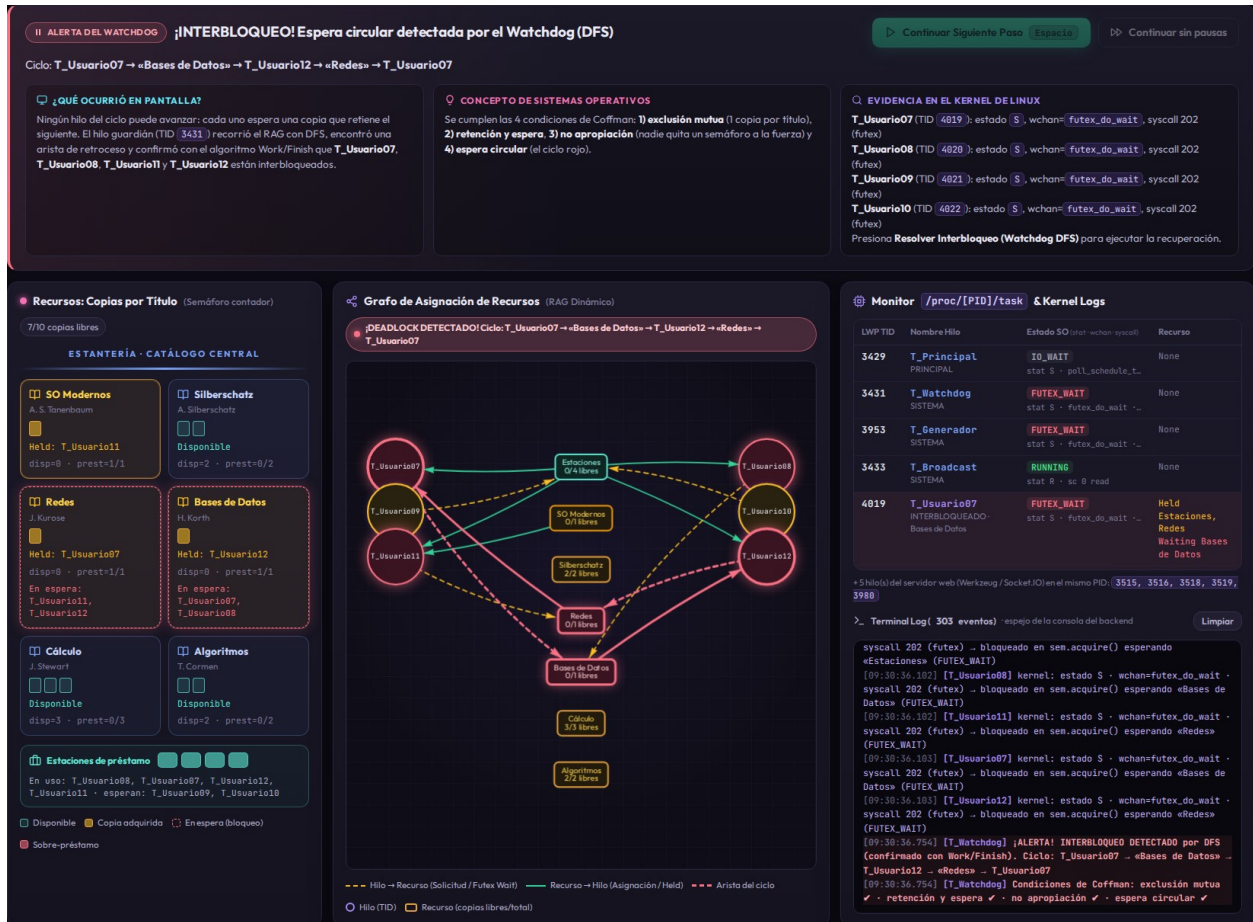


Figura 9. Interbloqueo espontáneo sin pausas: alerta del watchdog y ciclo resaltado mientras los hilos duermen en futex.

8. Evidencia en el sistema operativo

Entorno de la ejecución de prueba

Las capturas se tomaron en Linux x86-64 (kernel 6.18, glibc **NPTL 2.39**, Python 3.11) mientras había un interbloqueo en pantalla, con `./scripts/evidencia.sh`. Para la entrega, el equipo debe repetir exactamente los mismos comandos en su WSL2 durante la demostración y adjuntar sus propias capturas; la forma de la salida será la misma.

8.1 Hilos del proceso con su estado en el kernel

`ps -L -o pid,lwp,nlwp,stat,pcpu,wchan:24,comm -p <PID>`: un solo PID con 15 LWP. Los hilos de préstamo, nombrados con `prctl`, duermen en `futex_do_wait`:

PID	LWP	NLWP	STAT	%CPU	WCHAN	COMMAND
3429	3429	15	Ssl	1.1	poll_schedule_timeout.co	python3
3429	3431	15	Ssl	0.0	futex_do_wait	T_Watchdog
3429	3433	15	Ssl	0.4	-	T_Broadcast
3429	3515	15	Ssl	0.0	futex_do_wait	python3
3429	3516	15	Ssl	0.0	futex_do_wait	python3
3429	3517	15	Ssl	0.0	hrtimer_nanosleep	python3
3429	3518	15	Ssl	0.0	wait_woken	python3
3429	3519	15	Rsl	0.1	-	python3
3429	3524	15	Ssl	0.0	futex_do_wait	T_Generador
3429	3589	15	Ssl	0.0	futex_do_wait	T_Usuario07
3429	3590	15	Ssl	0.0	futex_do_wait	T_Usuario08
3429	3591	15	Ssl	0.0	futex_do_wait	T_Usuario09
3429	3592	15	Ssl	0.0	futex_do_wait	T_Usuario10
3429	3593	15	Ssl	0.0	futex_do_wait	T_Usuario11
3429	3594	15	Ssl	0.0	futex_do_wait	T_Usuario12

8.2 Detalle leído de /proc/<PID>/task

TID	COMM	STATE	WCHAN	SYSCALL(nr)
3429	python3	S	poll_schedule_timeout.constprop.0	7
3431	T_Watchdog	S	futex_do_wait	202
3433	T_Broadcast	S	hrtimer_nanosleep	230
3515	python3	S	futex_do_wait	202
3516	python3	S	futex_do_wait	202
3517	python3	S	hrtimer_nanosleep	230
3518	python3	S	wait_woken	45
3519	python3	S	futex_do_wait	202
3524	T_Generador	S	futex_do_wait	202
3589	T_Usuario07	S	futex_do_wait	202
3590	T_Usuario08	S	futex_do_wait	202
3591	T_Usuario09	S	futex_do_wait	202
3592	T_Usuario10	S	futex_do_wait	202
3593	T_Usuario11	S	futex_do_wait	202
3594	T_Usuario12	S	futex_do_wait	202

Nota: syscall 202 = futex en x86_64 (hilo dormido en `sem.acquire()/Lock/Condition`).

Los hilos `python3` sin nombre propio pertenecen al servidor web (Werkzeug / Socket.IO) y comparten el mismo PID. `ps -eLf` muestra las mismas 15 filas con `NLWP = 15`:

```

UID      PID  PPID   LWP  C  NLWP STIME TTY          TIME CMD
root    3429    1   3429   1    15 09:29 ?        00:00:00 python3 app.py --puerto 5000
root    3429    1   3431   0    15 09:29 ?        00:00:00 python3 app.py --puerto 5000
root    3429    1   3433   0    15 09:29 ?        00:00:00 python3 app.py --puerto 5000
root    3429    1   3515   0    15 09:29 ?        00:00:00 python3 app.py --puerto 5000
root    3429    1   3516   0    15 09:29 ?        00:00:00 python3 app.py --puerto 5000
...

```

8.3 Bitácora (WAL) con el TID de cada evento

```

09:29:15.544|pid=3429|tid=3528|T_Usuario04|BEGIN|ISBN...0004-2,ISBN...0001-1
09:29:15.548|pid=3429|tid=3528|T_Usuario04|ADQUIRIDO|ISBN...0004-2
09:29:17.918|pid=3429|tid=3528|T_Usuario04|ADQUIRIDO|ISBN...0001-1
09:29:17.919|pid=3429|tid=3528|T_Usuario04|COMMIT|ISBN...0004-2,ISBN...0001-1
09:29:20.210|pid=3429|tid=3528|T_Usuario04|DEVUELTO|ISBN...0001-1
09:29:20.211|pid=3429|tid=3528|T_Usuario04|DEVUELTO|ISBN...0004-2
09:29:32.525|pid=3429|tid=3431|T_Watchdog|DEADLOCK|T_Usuario11 → «Bases de Datos» → T_Usuario12 → «Redes» →
T_Usuario...
09:29:43.944|pid=3429|tid=3593|T_Usuario11|ABORT|víctima del watchdog; esperaba «Bases de Datos»
09:29:43.946|pid=3429|tid=3593|T_Usuario11|ROLLBACK|ISBN...0001-1
09:29:43.947|pid=3429|tid=3593|T_Usuario11|ROLLBACK|ESTACIONES

```

8.4 Otros comandos de verificación

```

ls -l /proc/<PID>/task/           # una entrada por hilo
cat /proc/<PID>/task/<TID>/wchan   # futex_do_wait para un hilo bloqueado
htop -p <PID>                     # tecla H: ver hilos
sudo strace -f -tt -e trace=futex -p <PID> # FUTEX_WAIT / FUTEX_WAKE en vivo
curl -s localhost:5000/api/estado # RAG y estado interno en JSON
python3 scripts/prueba_carga.py    # integridad bajo carga

```

9. Entregable 3 — Repositorio, reproducción y demostración

9.1 Estructura del repositorio

```
Proyecto2_BiblioSync/
├── app.py                servidor Flask-SocketIO (modo threading) + T_Broadcast
├── run.sh                crea .env, instala dependencias y ejecuta
├── requirements.txt      Flask, Flask-SocketIO, simple-websocket
├── README.md            instrucciones completas para WSL2 / VM
├── motor/               catalogo, recursos, transacciones, generador, rag,
│                       watchdog, pasos, explicaciones, kernel, bitacora
├── templates/index.html panel
├── static/              css, js, vendor (vis-network, socket.io), fuentes: sin internet
├── scripts/             evidencia.sh, prueba_carga.py
├── bitacora/            prestamos.wal (se genera al ejecutar)
├── evidencias/          salidas de evidencia.sh y prueba_carga.py
└── docs/               este documento, página de la Wiki e imágenes
```

9.2 Reproducir en WSL2

```
sudo apt update && sudo apt install -y python3 python3-venv python3-pip http strace curl
cp -r /mnt/c/Users/<usuario>/Downloads/Proyecto2_BiblioSync ~/ # fsync más rápido en ~/
cd ~/Proyecto2_BiblioSync
./run.sh                # o: python3 -m venv .env && . .env/bin/activate
                        # pip install -r requirements.txt && python3 app.py
# abrir http://localhost:5000 en el navegador de Windows
```

9.3 Guion de la demostración en vivo

Tabla 10. Guion sugerido (10 a 12 minutos).

#	Acción	Qué se demuestra
1	<code>./run.sh</code> y en otra terminal <code>ps -elf grep app.py</code>	Proceso con varios LWP; banner con el PID.
2	Modo Explicación + Transacciones Normales (3 pasos)	Semáforos, 2PL, futex; orden por ISBN sin ciclos.
3	Durante una pausa: <code>ps -L -o pid,lwp,stat,wchan:20,comm -p <PID></code>	Hilos <code>T_UsuarioNN</code> en <code>futex_do_wait</code> .
4	Provocar Interbloqueo → Paso 1 → Paso 2	Espera circular real, ciclo rojo, 4 condiciones de Coffman.
5	<code>./scripts/evidencia.sh</code> con el ciclo en pantalla	Evidencia externa: <code>/proc</code> , <code>ps</code> , <code>top -H</code> , JSON del RAG.
6	Continuar / Resolver → Paso 3	Víctima, ROLLBACK, todos terminan, integridad OK.

#	Acción	Qué se demuestra
7	Protección OFF → Probar Condición de Carrera; luego ON	Sobre-préstamo vs. 3 préstamos exactos.
8	<code>python3 scripts/prueba_carga.py</code>	Integridad bajo carga con 100 hilos.

9.4 Publicación en la Wiki

La guía indica que el proyecto debe añadirse a la Wiki. En [docs/wiki/Proyecto2-Motor-Transacciones.md](#) se incluye la página lista para copiar (compatible con Docusaurus y con la Wiki de GitHub), con las imágenes en [docs/img/](#).

10. Conclusiones

1. Los hilos de Python en Linux son hilos NPTL reales: cada solicitud de préstamo apareció en `ps -elf` y en `/proc/[pid]/task` con TID propio y, cuando esperaba una copia, en estado S dormido en `futex_do_wait`, lo que confirma que la espera la gestiona el kernel y no un ciclo de espera activa.
2. El GIL no protege las operaciones compuestas: el patrón verificar-y-actuar con una E/S real entre la lectura y la escritura produjo 12 préstamos de 3 copias en el panel y más de diez mil violaciones en la prueba de carga. Un `threading.Semaphore` contador eliminó por completo el problema (0 violaciones en 20 000 operaciones).
3. El semáforo contador es el mecanismo natural para recursos con N instancias idénticas, como las copias de un libro; el Lock se reservó para proteger las estructuras internas de las que se deriva el RAG.
4. Imponer un orden global de adquisición (ISBN) previene el interbloqueo; basta con respetar el orden de la cola para que la espera circular surja por sí sola, lo que evidencia que las cuatro condiciones de Coffman se cumplen simultáneamente en un sistema real.
5. Construir el RAG desde el estado real y recorrerlo con DFS permite detectar el ciclo en tiempo $O(V + E)$; con recursos de varias instancias es indispensable confirmar con el algoritmo Work/Finish para evitar falsos positivos.
6. La recuperación por víctima con rollback basado en bitácora resolvió el interbloqueo sin reiniciar el sistema, y el reintento diferido en orden de ISBN evitó la inanición: todas las solicitudes terminaron con el inventario íntegro.

11. Referencias

- Coffman, E. G., Elphick, M. J., & Shoshani, A. (1971). System deadlocks. *ACM Computing Surveys*, 3(2), 67–78. <https://doi.org/10.1145/356586.356588>
- Dijkstra, E. W. (1965). *Cooperating sequential processes* (EWD-123). Technological University Eindhoven.
- Drepper, U. (2011). *Futexes are tricky*. Red Hat, Inc.
- Drepper, U., & Molnar, I. (2005). *The Native POSIX Thread Library for Linux*. Red Hat, Inc.
- Eswaran, K. P., Gray, J. N., Lorie, R. A., & Traiger, I. L. (1976). The notions of consistency and predicate locks in a database system. *Communications of the ACM*, 19(11), 624–633. <https://doi.org/10.1145/360363.360369>
- Fette, I., & Melnikov, A. (2011). *The WebSocket protocol* (RFC 6455). Internet Engineering Task Force. <https://doi.org/10.17487/RFC6455>
- Franke, H., Russell, R., & Kirkwood, M. (2002). Fuss, futexes and furwocks: Fast userlevel locking in Linux. En *Proceedings of the Ottawa Linux Symposium* (pp. 479–495).
- Grinberg, M. (s. f.). *Flask-SocketIO documentation*. <https://flask-socketio.readthedocs.io/>
- Havender, J. W. (1968). Avoiding deadlock in multitasking systems. *IBM Systems Journal*, 7(2), 74–84. <https://doi.org/10.1147/sj.72.0074>
- Hoare, C. A. R. (1974). Monitors: An operating system structuring concept. *Communications of the ACM*, 17(10), 549–557. <https://doi.org/10.1145/355620.361161>
- Holt, R. C. (1972). Some deadlock properties of computer systems. *ACM Computing Surveys*, 4(3), 179–196. <https://doi.org/10.1145/356603.356607>
- Kerrisk, M. (s. f.). *The Linux man-pages project: proc(5), futex(2), clone(2), ps(1)*. <https://man7.org/linux/man-pages/>
- Python Software Foundation. (s. f.). *threading — Thread-based parallelism*. <https://docs.python.org/3/library/threading.html>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10.^a ed.). Wiley.
- Stallings, W. (2018). *Operating systems: Internals and design principles* (9.^a ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4.^a ed.). Pearson.
- Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2), 146–160. <https://doi.org/10.1137/0201010>
- vis.js community. (s. f.). *vis-network documentation*. <https://visjs.github.io/vis-network/docs/network/>

Anexo A. Lista de revisión del docente

Tabla 11. Criterios de la hoja de trabajo y dónde se cumplen.

Criterio del docente	Cumplimiento	Sección
El escenario genera contención real (no es trivial ni ya resuelto).	Títulos de 1 copia pedidos en préstamos combinados desde varias sucursales; órdenes cruzados.	4.1, 4.5
El plan de sincronización es coherente con el mecanismo elegido.	Semáforo contador por título (recurso de N copias) + Lock solo para el estado del RAG.	4.2, 5.4
El diseño del deadlock fuerza una espera circular genuina, no actuada.	Orden de la cola + planificador + E/S real; sin <code>sleep()</code> entre adquisiciones; ocurre también sin pausas.	4.5, 5.6, 7.4
El frontend refleja el estado real del backend, no datos de ejemplo.	WebSocket cada 250 ms con <code>snapshot()</code> del motor y <code>/proc/self/task</code> .	4.7, 5.6
Alcance viable en el tiempo restante del curso.	Cronograma de 5 semanas; motor y panel ya funcionales.	4.8
Evidencia externa: <code>ps -elf, /proc, htop, strace</code> .	<code>scripts/evidencia.sh</code> y capturas del capítulo 8.	8
README para reproducir en WSL/VM y demo del ciclo completo.	README.md, <code>run.sh</code> y guion de demostración.	9